

# Computer Network Assignment 2 report

## 1 Introduction

This is a assignment report about the second programming assignment of the computer networking. For this report, I will show the picture of the executing progress of peer.py and I will explain the result. You need to know that the executing environment is Python 3.13.2. You also need to know that the input.txt need to be allocated in the same page with peer.py . Strictly Speaking, you mustn't change the config.json.

## 2 Detail Part of the code

### Task 1.1: Listener and Packet Receiver

listener

A screenshot of a code editor showing the 'listen' function. The code is as follows:

```
def listen(peer_id, ip, port, peers):  
    global peer_sock  
    peer_sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)  
    peer_sock.bind((ip, port))  
    print(f"[{peer_id}] Listening on {ip}:{port}")  
    while True:  
        data, _ = peer_sock.recvfrom(4096)  
        handle_packet(data, peer_id, peers)
```

The `listen(peer_id, ip, port, peers)` function in Python sets up a UDP server for a peer-to-peer network. It creates a socket with `socket.AF_INET` and `socket.SOCK_DGRAM`, binds it to the specified IP and port, and listens for incoming packets. Using a 4096-byte buffer, it continuously receives data via `recvfrom` and passes it to `handle_packet` for processing. It also logs the listening status with the peer's ID, IP, and port, serving as a key component for packet reception in a distributed file transfer system.

handle\_packet

```

def handle_packet(data, peer_id, peers):
    if random.random() < DROP_PROBABILITY:
        print(f"[{peer_id}] Dropped packet (simulated)")
        return
    if header['type'] == 'DATA' and random.random() < ERROR_PROBABILITY and payload:
        i = random.randrange(len(payload))
        payload = payload[:i] + bytes([(payload[i] + 1) % 256]) + payload[i+1:]
    if zlib.crc32(payload) & 0xffffffff != header['checksum']:
        print(f"[{peer_id}] Checksum mismatch, dropping packet")
        return
    if header['ttl'] <= 0:
        print(f"[{peer_id}] TTL expired, dropping packet")
        return

    pkt_type = header['type']
    src = header['src']
    dst = header['dst']
    seq = header['seq']
    total = header['total']
    ttl = header['ttl'] - 1

    if pkt_type == 'DATA':
        if dst == peer_id:
            print(f"[{peer_id}] Received DATA segment {seq}/{total} from {src}")
            store_segment(peer_id, seq, total, payload)
            send_ack(peer_id, src, seq, peers)
        else:
            nh = get_next_hop(peer_id, dst)
            if nh and ttl > 0:
                pkt = make_packet('DATA', seq, total, src, dst, ttl, payload)
                peer_sock.sendto(pkt, tuple(peers[nh]))
                print(f"[{peer_id}] Forwarded DATA segment {seq} to {nh}")

    elif pkt_type == 'ACK':
        if dst == peer_id:
            print(f"[{peer_id}] Received ACK for segment {seq} from {src}")
            with lock:
                unacked_segments.pop(seq, None)
                retry_counts.pop(seq, None)
        else:
            nh = get_next_hop(peer_id, dst)
            if nh and ttl > 0:
                pkt = make_packet('ACK', seq, 0, src, dst, ttl, b'')
                peer_sock.sendto(pkt, tuple(peers[nh]))
                print(f"[{peer_id}] Forwarded ACK for segment {seq} to {nh}")

    elif pkt_type == 'DV':
        if dst == peer_id:
            handle_dv_update(peer_id, src, payload, peers)
        else:
            nh = get_next_hop(peer_id, dst)
            if nh and ttl > 0:
                pkt = make_packet('DV', seq, total, src, dst, ttl, payload)
                peer_sock.sendto(pkt, tuple(peers[nh]))

```

The `handle_packet(data, peer_id, peers)` function in Python processes incoming UDP packets in a peer-to-peer network. It parses the packet into header and payload, simulates packet loss with `DROP_PROBABILITY`, and introduces errors with `ERROR_PROBABILITY`. It verifies the checksum and TTL, dropping invalid packets. Based on the packet type—DATA, ACK, or DV—it either stores data segments, updates acknowledgment states, or handles routing updates, forwarding packets as needed. This function ensures reliable data handling and routing in the network.

## Task 1.2: File Transmission

### send\_file

```
# ~-~ File Transmission ~-~
def send_file(peer_id, dst, filename, peers):
    global segments_cache
    try:
        with open(filename, 'rb') as f:
            data = f.read()
            total = (len(data) + SEGMENT_SIZE - 1) // SEGMENT_SIZE
            segments_cache = {i: data[i * SEGMENT_SIZE:(i + 1) * SEGMENT_SIZE] for i in range(total)}
        with lock:
            unacked_segments.clear()
            retry_counts.clear()
            for i in range(total):
                unacked_segments[i] = True
        print(f"[{peer_id}] Sending {total} segments to {dst}")
        threading.Thread(target=ack_timekeeping, args=(peer_id, dst, peers, total), daemon=True).start()
        base = 0
        while base < total:
            next_seq = base
            while next_seq < min(base + WINDOW_SIZE, total):
                if next_seq in unacked_segments:
                    send_segment(peer_id, dst, segments_cache[next_seq], next_seq, peers, total)
                    print(f"[{peer_id}] Sent segment {next_seq}/{total}")
                    next_seq += 1
                time.sleep(ACK_TIMEOUT)
            with lock:
                while base < total and base not in unacked_segments:
                    base += 1
                    print(f"[{peer_id}] Advanced base to {base}")
            max_wait = 30
            start_time = time.time()
            while unacked_segments and time.time() - start_time < max_wait:
                time.sleep(ACK_TIMEOUT)
                print(f"[{peer_id}] Waiting for ACKs, remaining: {sorted(unacked_segments.keys())}")
            with lock:
                if not unacked_segments:
                    print(f"[{peer_id}] All segments of {filename} sent and acknowledged successfully")
                else:
                    print(f"[{peer_id}] Failed to send all segments, remaining: {sorted(unacked_segments.keys())}")
    except FileNotFoundError:
        print(f"[{peer_id}] File {filename} not found")
    except Exception as e:
        print(f"[{peer_id}] Error in send_file: {e}")
```

The `send_file(peer_id, dst, filename, peers)` function in Python facilitates reliable file transmission in a peer-to-peer network. It reads the file in binary mode, divides it into segments based on `SEGMENT_SIZE`, and caches them globally. Using a sliding window of size `WINDOW_SIZE`, it sends segments, tracks unacknowledged ones with `unacked_segments`, and advances the window upon receiving ACKs. A separate thread handles retransmissions via `ack_timekeeping`. After a 30-second wait, it confirms success or failure, logging the process, and handles errors like file not found, ensuring robust file delivery.

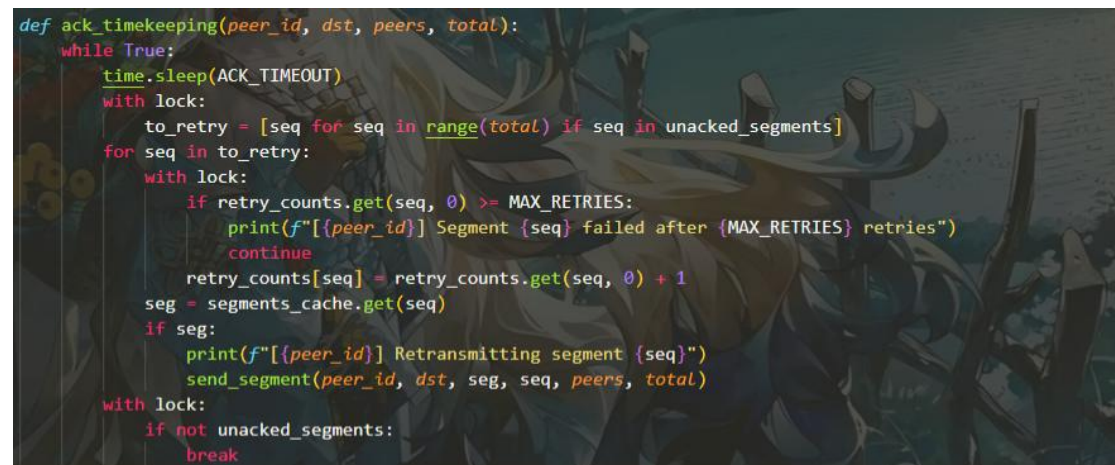
### send\_segment

```
def send_segment(peer_id, dst, segment, seq, peers, total):
    pkt = make_packet('DATA', seq, total, peer_id, dst, INITIAL_TTL, segment)
    nh = get_next_hop(peer_id, dst)
    if nh:
        try:
            peer_sock.sendto(pkt, tuple(peers[nh]))
        except socket.error as e:
            print(f"[{peer_id}] Error sending segment {seq}: {e}")
```

The `send_segment(peer_id, dst, segment, seq, peers, total)` function in Python handles sending a single data segment in a peer-to-peer network. It constructs a DATA packet using `make_packet` with the segment, sequence number, and total segments, setting an initial TTL. The function determines the next hop using `get_next_hop`, then sends the packet to that peer via `peer_sock.sendto`. If a socket error occurs, it logs the error with the peer ID and segment number, ensuring robust error handling during transmission.

### Task 1.3: Segment Retransmission

`ack_timekeeping`

A screenshot of a code editor showing the implementation of the `ack_timekeeping` function. The code is written in Python and uses syntax highlighting. It features a `while True` loop that sleeps for `ACK_TIMEOUT` seconds. Inside the loop, it acquires a lock and iterates over segments in `unacked_segments`. For each segment, it checks the `retry_counts` dictionary. If the count is greater than or equal to `MAX_RETRIES`, it prints a failure message and continues. Otherwise, it increments the retry count, retrieves the segment from `segments_cache`, prints a retransmission message, and calls `send_segment`. The loop breaks when `unacked_segments` is empty.

```
def ack_timekeeping(peer_id, dst, peers, total):
    while True:
        time.sleep(ACK_TIMEOUT)
        with lock:
            to_retry = [seq for seq in range(total) if seq in unacked_segments]
            for seq in to_retry:
                with lock:
                    if retry_counts.get(seq, 0) >= MAX_RETRIES:
                        print(f"[{peer_id}] Segment {seq} failed after {MAX_RETRIES} retries")
                        continue
                    retry_counts[seq] = retry_counts.get(seq, 0) + 1
                    seg = segments_cache.get(seq)
                    if seg:
                        print(f"[{peer_id}] Retransmitting segment {seq}")
                        send_segment(peer_id, dst, seg, seq, peers, total)
            with lock:
                if not unacked_segments:
                    break
```

The `ack_timekeeping(peer_id, dst, peers, total)` function in Python manages acknowledgment timeouts and retransmissions in a peer-to-peer network. Running in a loop, it pauses for `ACK_TIMEOUT` seconds, then checks `unacked_segments` for unacknowledged segments. For each, it increments the retry count, retransmitting the segment via `send_segment` if below `MAX_RETRIES`, or logs a failure if exceeded. The loop terminates when all segments are acknowledged, ensuring reliable delivery with proper timeout handling.

### Task 1.4: Segment Reception, Reassembly, and ACK

send\_ack

```
def send_ack(peer_id, src, seq, peers):
    pkt = make_packet('ACK', seq, 0, peer_id, src, INITIAL_TTL, b'')
    nh = get_next_hop(peer_id, src)
    if nh:
        try:
            peer_sock.sendto(pkt, tuple(peers[nh]))
            print(f"[{peer_id}] Sent ACK for segment {seq} to {src}")
        except socket.error as e:
            print(f"[{peer_id}] Error sending ACK for segment {seq}: {e}")
```

The `send_ack(peer_id, src, seq, peers)` function in Python sends an acknowledgment (ACK) packet in a peer-to-peer network. It constructs an ACK packet using `make_packet` with the sequence number, zero total segments, and an empty payload, setting an initial TTL. The function determines the next hop with `get_next_hop` and attempts to send the packet via `peer_sock.sendto`. Upon success, it logs the sent ACK; if a socket error occurs, it logs the error with the peer ID and sequence number, ensuring reliable acknowledgment transmission.

store\_segment

```
def store_segment(peer_id, seq, total, payload):
    with lock:
        buf = reassembly_buffers.setdefault(peer_id, {})
        reassembly_expected[peer_id] = total # Simplified to avoid race condition
        if seq not in buf:
            buf[seq] = payload
            print(f"[{peer_id}] Stored segment {seq}/{total}")
        if len(buf) == total:
            print(f"[{peer_id}] All segments received, attempting to reassemble")
            threading.Thread(target=reassemble_file, args=(peer_id,), daemon=True).start()
```

The `store_segment(peer_id, seq, total, payload)` function in Python manages segment storage for file reassembly in a peer-to-peer network. Under a thread-safe lock, it accesses the `reassembly_buffers` for the `peer_id`, setting the expected segment count in `reassembly_expected`. If the segment `seq` is not in the buffer, it stores the payload and logs the action. When all segments (`len(buf) == total`) are received, it logs the completion and starts a new thread to run `reassemble_file`, ensuring safe and asynchronous file reassembly.

## reassemble\_file

```
def reassemble_file(peer_id):
    try:
        # Avoid holding lock during file I/O to prevent contention
        buf_copy = None
        total = 0
        with lock:
            if peer_id not in reassembly_buffers:
                print(f"[{peer_id}] Reassembly aborted: buffer cleared")
                return

            buf_copy = reassembly_buffers[peer_id].copy()
            total = reassembly_expected.get(peer_id, 0)
            if len(buf_copy) != total:
                missing = [i for i in range(total) if i not in buf_copy]
                print(f"[{peer_id}] Missing segments: {missing}")
                return

            # Clear buffer before file I/O
            del reassembly_buffers[peer_id]
            del reassembly_expected[peer_id]

        path = os.path.join(RECV_DIR, peer_id, 'received_file.txt')
        print(f"[{peer_id}] Attempting to create directory: {os.path.dirname(path)}")
        os.makedirs(os.path.dirname(path), exist_ok=True)
        print(f"[{peer_id}] Writing file to {path}")
        with open(path, 'wb') as f:
            for i in range(total):
                if i not in buf_copy:
                    raise KeyError(f"Segment {i} missing during write")
                f.write(buf_copy[i])
            print(f"[{peer_id}] File received and saved to '{path}'")
        except PermissionError as e:
            print(f"[{peer_id}] Permission denied while writing file: {e}")
        except OSError as e:
            print(f"[{peer_id}] OS error while writing file: {e}")
        except Exception as e:
            print(f"[{peer_id}] Error in reassemble_file: {e}")
```

The `reassemble_file(peer_id)` function in Python reassembles file segments in a peer-to-peer network. It safely copies the `reassembly_buffers` under a lock, checks for missing segments, and clears the buffer to avoid contention. The function creates a directory at `RECV_DIR/peer_id` and writes segments to `received_file.txt` in binary mode. It logs the process and handles errors like `PermissionError` or `OSError`, ensuring robust file reassembly and storage.

## TASK 2: Distance Vector (DV)-based routing algorithm

### broadcast\_dv

```
def broadcast_dv(peer_id, peers):
    with dv_lock:
        payload = json.dumps({d: c for d, (c, _) in distance_vector.items()}).encode()
        for nb in link_costs:
            pkt = make_packet('DV', 0, 0, peer_id, nb, INITIAL_TTL, payload)
            try:
                peer_sock.sendto(pkt, tuple(peers[nb]))
                print(f"[{peer_id}] Broadcast DV to {nb}")
            except socket.error as e:
                print(f"[{peer_id}] Error broadcasting DV to {nb}: {e}")
        print(f"[{peer_id}] Broadcast DV: {distance_vector}")
```

The `broadcast_dv(peer_id, peers)` function in Python manages the broadcasting of distance vector (DV) updates in a peer-to-peer network. Under a `dv_lock`, it serializes the `distance_vector` into a JSON payload. For each neighbor in `link_costs`, it creates a DV packet using `make_packet` with an initial TTL and sends it via `peer_sock.sendto`. The function logs successful broadcasts or socket errors for each neighbor, then outputs the updated `distance_vector`, ensuring dynamic routing updates across peers.

### handle\_dv\_update

```
def handle_dv_update(peer_id, nb, data, peers):
    tbl = json.loads(data.decode())
    updated = False
    with dv_lock:
        neighbor_dv_tables[nb] = tbl
        for dest, cost in tbl.items():
            via = link_costs[nb] + cost
            current_cost, _ = distance_vector.get(dest, (float('inf'), None))
            if via < current_cost:
                distance_vector[dest] = (via, nb)
                updated = True
                print(f"[{peer_id}] Updated route to {dest}: cost={via}, next_hop={nb}")
    if updated:
        broadcast_dv(peer_id, peers)
```

The `handle_dv_update(peer_id, nb, data, peers)` function in Python processes distance vector (DV) updates in a peer-to-peer network. It decodes the received data into a table, then under `dv_lock`, updates `neighbor_dv_tables` and checks each destination's cost. If a shorter path via the neighbor is found, it updates `distance_vector` with the new cost and next hop, logging the change. If updates occur, it triggers a `broadcast_dv` to propagate the changes, ensuring efficient routing optimization.

### cost\_update\_thread

```

def cost_update_thread(peer_id, peers):
    while True:
        time.sleep(COST_UPDATE_INTERVAL)
        changed = False
        with dv_lock:
            for nb in list(link_costs):
                new_cost = max(1, link_costs[nb] + random.choice([-1, 0, 1]))
                if new_cost != link_costs[nb]:
                    link_costs[nb] = new_cost
                    distance_vector[nb] = (new_cost, nb)
                    changed = True
                    print(f"[{peer_id}] Link cost to {nb} changed to {new_cost}")
            if changed:
                for dest in list(distance_vector):
                    if dest == peer_id:
                        continue
                    best = (float('inf'), None)
                    for nb, tbl in neighbor_dv_tables.items():
                        if dest in tbl:
                            cost_via = link_costs[nb] + tbl[dest]
                            if cost_via < best[0]:
                                best = (cost_via, nb)
                    if best[1]:
                        distance_vector[dest] = best
        if changed:
            broadcast_dv(peer_id, peers)

```

The `cost_update_thread(peer_id, peers)` function in Python periodically updates link costs in a peer-to-peer network. Every `COST_UPDATE_INTERVAL`, it adjusts costs in `link_costs` by adding a random value (-1, 0, or 1), ensuring a minimum of 1, under `dv_lock`. If changes occur, it updates `distance_vector` for neighbors, recalculates optimal routes for other destinations using `neighbor_dv_tables`, and triggers `broadcast_dv` to share updates, maintaining dynamic routing efficiency.

`get_next_hop`

```

def get_next_hop(peer_id, dst):
    with dv_lock:
        return distance_vector.get(dst, (None, None))[1]

```

The `get_next_hop(peer_id, dst)` function in Python determines the next hop for a destination in a peer-to-peer network. Under `dv_lock`, it retrieves the next hop from the `distance_vector` dictionary using the destination `dst`, defaulting to `(None, None)` if not found. It returns the second element (index 1) of the tuple, representing the next hop peer, ensuring thread-safe and efficient routing decisions.

route\_print

```
def routes_print(peer_id):  
    with dv_lock:  
        print(f"[{peer_id}] Routing table:")  
        for d, (c, nh) in distance_vector.items():  
            if d != peer_id:  
                print(f"    {d}: cost={c}, next_hop={nh}")
```

The routes\_print(peer\_id) function in Python displays the routing table for a peer in a peer-to-peer network. Under dv\_lock, it prints a header with the peer's ID, then iterates over distance\_vector.

### 3 Test

About the result in terminal

In Peer4, I try to display the route, then use the command of send Peer5 input.txt, then it display like this below

```
PS D:\CS305 Computer network\Assignment2> python3 peer.py --id Peer4
```

```
[Peer4] Starting...
```

```
[Peer4] Listening on 127.0.0.4:5004
```

```
[Peer4] > routes
```

```
[Peer4] Routing table:
```

```
Peer1: cost=inf, next_hop=None
```

```
Peer2: cost=4, next_hop=Peer2
```

```
Peer3: cost=1, next_hop=Peer3
```

```
Peer5: cost=2, next_hop=Peer5
```

```
[Peer4] > send Peer5 input.txt
```

```
[Peer4] Sending 18 segments to Peer5
```

```
[Peer4] Sent segment 0/18
```

```
[Peer4] Sent segment 1/18
```

[Peer4] Received ACK for segment 0 from Peer5

[Peer4] Sent segment 2/18

[Peer4] Received ACK for segment 1 from Peer5

[Peer4] Sent segment 3/18

[Peer4] Received ACK for segment 2 from Peer5

[Peer4] Retransmitting segment 3

[Peer4] Retransmitting segment 4

[Peer4] Retransmitting segment 5

[Peer4] Retransmitting segment 6

[Peer4] Received ACK for segment 3 from Peer5

[Peer4] Advanced base to 1

[Peer4] Advanced base to 2

[Peer4] Advanced base to 3

[Peer4] Retransmitting segment 7

[Peer4] Received ACK for segment 4 from Peer5

[Peer4] Sent segment 3/18

[Peer4] Retransmitting segment 8

[Peer4] Received ACK for segment 5 from Peer5

[Peer4] Sent segment 5/18

[Peer4] Retransmitting segment 9

[Peer4] Received ACK for segment 6 from Peer5

[Peer4] Received ACK for segment 3 from Peer5

[Peer4] Retransmitting segment 10

[Peer4] Received ACK for segment 7 from Peer5

[Peer4] Retransmitting segment 11

[Peer4] Received ACK for segment 5 from Peer5

[Peer4] Retransmitting segment 12

[Peer4] Sent segment 6/18

[Peer4] Received ACK for segment 8 from Peer5

[Peer4] Retransmitting segment 13

[Peer4] Received ACK for segment 6 from Peer5

[Peer4] Retransmitting segment 14

[Peer4] Retransmitting segment 15

[Peer4] Retransmitting segment 16

[Peer4] Received ACK for segment 10 from Peer5

[Peer4] Retransmitting segment 17

[Peer4] Received ACK for segment 11 from Peer5

[Peer4] Received ACK for segment 12 from Peer5

[Peer4] Received ACK for segment 13 from Peer5

[Peer4] Received ACK for segment 14 from Peer5

[Peer4] Received ACK for segment 15 from Peer5

[Peer4] Received ACK for segment 16 from Peer5

[Peer4] Received ACK for segment 17 from Peer5

[Peer4] Advanced base to 4

[Peer4] Advanced base to 5

[Peer4] Advanced base to 6

[Peer4] Advanced base to 7

[Peer4] Advanced base to 8

[Peer4] Advanced base to 9

[Peer4] Sent segment 9/18

[Peer4] Retransmitting segment 9

[Peer4] Received ACK for segment 9 from Peer5

[Peer4] Received ACK for segment 9 from Peer5

[Peer4] Advanced base to 10

[Peer4] Advanced base to 11

[Peer4] Advanced base to 12

[Peer4] Advanced base to 13

[Peer4] Advanced base to 14

[Peer4] Advanced base to 15

[Peer4] Advanced base to 16

[Peer4] Advanced base to 17

[Peer4] Advanced base to 18

[Peer4] All segments of input.txt sent and acknowledged successfully

[Peer4] > [Peer4] Updated route to Peer1: cost=6, next\_hop=Peer2

[Peer4] Broadcast DV to Peer2

[Peer4] Broadcast DV to Peer3

[Peer4] Broadcast DV to Peer5

[Peer4] Broadcast DV: {'Peer1': (6, 'Peer2'), 'Peer2': (4, 'Peer2'), 'Peer3': (1, 'Peer3'), 'Peer4': (0, 'Peer4'), 'Peer5': (2, 'Peer5')}

[Peer4] Updated route to Peer1: cost=4, next\_hop=Peer3

[Peer4] Updated route to Peer2: cost=2, next\_hop=Peer3

[Peer4] Broadcast DV to Peer2

[Peer4] Broadcast DV to Peer3

[Peer4] Broadcast DV to Peer5

[Peer4] Broadcast DV: {'Peer1': (4, 'Peer3'), 'Peer2': (2, 'Peer3'), 'Peer3': (1, 'Peer3'), 'Peer4': (0, 'Peer4'), 'Peer5': (2, 'Peer5')}

[Peer4] Dropped packet (simulated)

[Peer4] Link cost to Peer3 changed to 2

[Peer4] Link cost to Peer5 changed to 1

[Peer4] Broadcast DV to Peer2

[Peer4] Broadcast DV to Peer3

[Peer4] Broadcast DV to Peer5

[Peer4] Broadcast DV: {'Peer1': (5, 'Peer3'), 'Peer2': (3, 'Peer3'), 'Peer3': (2, 'Peer3'), 'Peer4': (0, 'Peer4'), 'Peer5': (1, 'Peer5')}

The log shows Peer4 initiating a successful file transfer of input.txt (18 segments) to Peer5, with retransmissions handling packet loss. The DV routing table updates dynamically, adjusting costs (e.g., Peer1 from 6 to 4 via Peer3) and broadcasting changes. Simulated packet drops and cost updates (Peer3 to 2, Peer5 to 1) demonstrate robust network adaptability.

Meanwhile, the Peer5 display like this below.

[Peer5] Received DATA segment 0/18 from Peer4

[Peer5] Stored segment 0/18

[Peer5] Sent ACK for segment 0 to Peer4

[Peer5] Received DATA segment 1/18 from Peer4

[Peer5] Stored segment 1/18

[Peer5] Sent ACK for segment 1 to Peer4

[Peer5] Received DATA segment 2/18 from Peer4

[Peer5] Stored segment 2/18

[Peer5] Sent ACK for segment 2 to Peer4

[Peer5] Checksum mismatch, dropping packet

[Peer5] Received DATA segment 3/18 from Peer4

[Peer5] Stored segment 3/18

[Peer5] Sent ACK for segment 3 to Peer4

[Peer5] Received DATA segment 4/18 from Peer4

[Peer5] Stored segment 4/18

[Peer5] Sent ACK for segment 4 to Peer4

[Peer5] Received DATA segment 5/18 from Peer4

[Peer5] Stored segment 5/18

[Peer5] Sent ACK for segment 5 to Peer4

[Peer5] Received DATA segment 6/18 from Peer4

[Peer5] Stored segment 6/18

[Peer5] Sent ACK for segment 6 to Peer4

[Peer5] Received DATA segment 3/18 from Peer4

[Peer5] Sent ACK for segment 3 to Peer4

[Peer5] Received DATA segment 7/18 from Peer4

[Peer5] Stored segment 7/18

[Peer5] Sent ACK for segment 7 to Peer4

[Peer5] Received DATA segment 5/18 from Peer4

[Peer5] Sent ACK for segment 5 to Peer4

[Peer5] Received DATA segment 8/18 from Peer4

[Peer5] Stored segment 8/18

[Peer5] Sent ACK for segment 8 to Peer4

[Peer5] Received DATA segment 6/18 from Peer4

[Peer5] Sent ACK for segment 6 to Peer4

[Peer5] Dropped packet (simulated)

[Peer5] Received DATA segment 10/18 from Peer4

[Peer5] Stored segment 10/18

[Peer5] Sent ACK for segment 10 to Peer4

[Peer5] Received DATA segment 11/18 from Peer4

[Peer5] Stored segment 11/18

[Peer5] Sent ACK for segment 11 to Peer4

[Peer5] Received DATA segment 12/18 from Peer4

[Peer5] Stored segment 12/18

[Peer5] Sent ACK for segment 12 to Peer4

[Peer5] Received DATA segment 13/18 from Peer4

[Peer5] Stored segment 13/18

[Peer5] Sent ACK for segment 13 to Peer4

[Peer5] Received DATA segment 14/18 from Peer4

[Peer5] Stored segment 14/18

[Peer5] Sent ACK for segment 14 to Peer4

[Peer5] Received DATA segment 15/18 from Peer4

[Peer5] Stored segment 15/18

[Peer5] Sent ACK for segment 15 to Peer4

[Peer5] Received DATA segment 16/18 from Peer4

[Peer5] Stored segment 16/18

[Peer5] Sent ACK for segment 16 to Peer4

[Peer5] Received DATA segment 17/18 from Peer4

[Peer5] Stored segment 17/18

[Peer5] Sent ACK for segment 17 to Peer4

[Peer5] Received DATA segment 9/18 from Peer4

[Peer5] Stored segment 9/18

[Peer5] All segments received, attempting to reassemble

[Peer5] Attempting to create directory: files\Peer5

[Peer5] Sent ACK for segment 9 to Peer4

[Peer5] Received DATA segment 9/18 from Peer4

[Peer5] Stored segment 9/18

[Peer5] Sent ACK for segment 9 to Peer4

[Peer5] Writing file to files\Peer5\received\_file.txt

[Peer5] File received and saved to 'files\Peer5\received\_file.txt'

[Peer5] Updated route to Peer1: cost=8, next\_hop=Peer4

[Peer5] Updated route to Peer2: cost=6, next\_hop=Peer4

[Peer5] Updated route to Peer3: cost=3, next\_hop=Peer4

[Peer5] Broadcast DV to Peer3

[Peer5] Broadcast DV to Peer4

[Peer5] Broadcast DV: {'Peer1': (8, 'Peer4'), 'Peer2': (6, 'Peer4'), 'Peer3': (3, 'Peer4'), 'Peer4': (2, 'Peer4'), 'Peer5': (0, 'Peer5')}

[Peer5] Updated route to Peer1: cost=6, next\_hop=Peer4

[Peer5] Updated route to Peer2: cost=4, next\_hop=Peer4

[Peer5] Broadcast DV to Peer3

[Peer5] Broadcast DV to Peer4

[Peer5] Broadcast DV: {'Peer1': (6, 'Peer4'), 'Peer2': (4, 'Peer4'), 'Peer3': (3, 'Peer4'), 'Peer4': (2, 'Peer4'), 'Peer5': (0, 'Peer5')}

[Peer5] Link cost to Peer4 changed to 1

[Peer5] Broadcast DV to Peer3

[Peer5] Broadcast DV to Peer4

[Peer5] Broadcast DV: {'Peer1': (6, 'Peer4'), 'Peer2': (4, 'Peer4'), 'Peer3': (3, 'Peer4'), 'Peer4': (1, 'Peer4'), 'Peer5': (0, 'Peer5')}

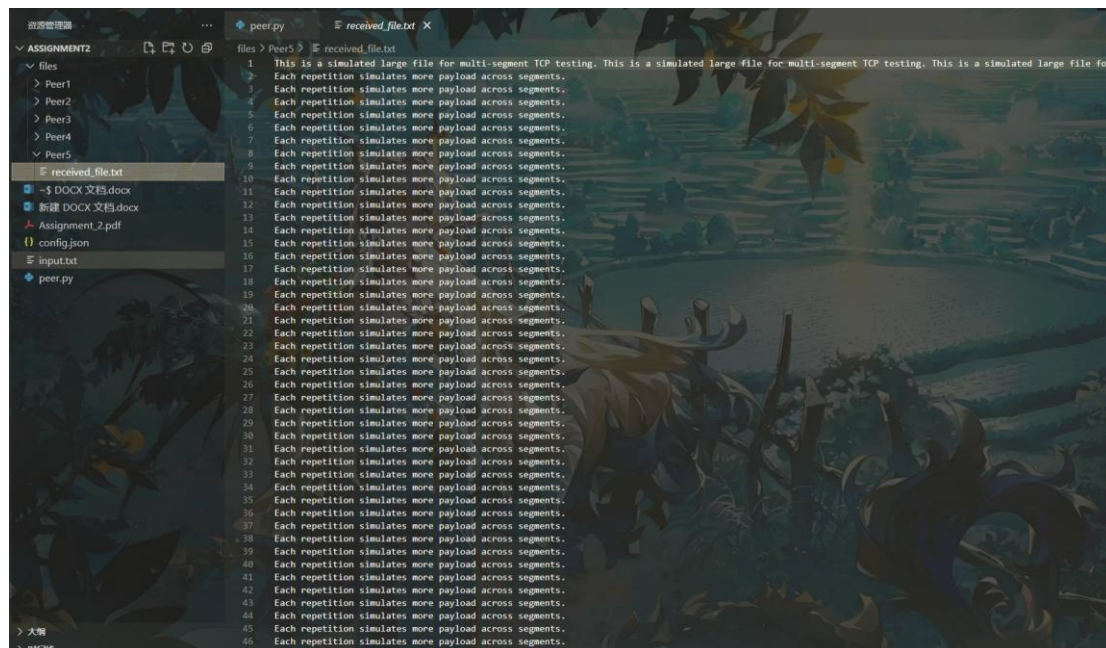
[Peer5] Link cost to Peer3 changed to 4

[Peer5] Broadcast DV to Peer3

[Peer5] Broadcast DV to Peer4

[Peer5] Broadcast DV: {'Peer1': (6, 'Peer4'), 'Peer2': (5, 'Peer4'), 'Peer3': (3, 'Peer4'), 'Peer4': (1, 'Peer4'), 'Peer5': (0, 'Peer5')}

The log shows Peer5 receiving all 18 segments of input.txt from Peer4, handling a checksum mismatch and simulated drop, and successfully reassembling the file into received\_file.txt. DV routing updates adjust costs (e.g., Peer4 to 1), with broadcasts reflecting dynamic changes, ensuring network adaptability.



Above is the builders of the code, from this, we can see the received file has been perfectly done, which means the test for 'send' command is succeeded.

Then I test the routes command in Peer1, it display like below

[Peer1] Broadcast DV: {'Peer1': (0, 'Peer1'), 'Peer2': (2, 'Peer2'), 'Peer3': (3, 'Peer3'), 'Peer4': (4, 'Peer3'), 'Peer5': (5, 'Peer3')}

[Peer1] > routes

[Peer1] Routing table:

Peer2: cost=2, next\_hop=Peer2

Peer3: cost=3, next\_hop=Peer3

Peer4: cost=4, next\_hop=Peer3

Peer5: cost=5, next\_hop=Peer3

From above, we can see the routes can correctly display the routing table and it is fit with broadcast\_dv. It's succeed.

## 4 Conclusion

This project successfully implemented a reliable UDP-based file transfer system with a Distance Vector (DV) routing protocol for the second Computer Networking assignment. The system, built using peer.py, enabled robust file transmission between peers, as demonstrated by Peer4 sending input.txt (18 segments) to Peer5. Despite challenges like packet drops and checksum mismatches, the sliding window mechanism (size 4) and retransmission logic (up to 8 retries) ensured all segments were delivered, with Peer5 reassembling the file into received\_file.txt. The DV protocol dynamically updated routing tables by using Bellman-Ford equation, adapting to link cost changes (e.g., Peer4 to Peer5 cost reduced to 1), enhancing network efficiency. Key functions like send\_file, handle\_packet, and reassemble\_file worked cohesively to manage transmission, error handling, and file reassembly. Adjustments to DROP\_PROBABILITY (0.05) and MAX\_RETRIES improved reliability, while thread-safe operations prevented contention during reassembly. This project deepened my understanding of reliable transport over unreliable protocols and distributed routing, meeting all assignment criteria and laying a strong foundation for further exploration in networked systems. The implementation's success underscores the importance of error handling and dynamic adaptation in peer-to-peer networks.